

UNIVERSITÉ D'AIX-MARSEILLE • BUT 2 INFORMATIQUE ARLES

R3.L01 — Représentations et codages des images

TP n°1 & TD n°1 : Conception de la classe GrayImage & Format PGM

Guide intégral d'explications et d'architectures basé sur le cours de **M. Éric Rémy**

Syntaxe 100% conforme au cours/TD, explications du POURQUOI & Code complet en annexes

Contrôle TD : 14/10/2026

Contrôle TP : 06/11/2026 (ou 21/10)

Rendu : 5ème séance (100% du code)

Qu'est-ce qu'une image matricielle ?

CM Diapo 24 / 87

- **Discrétisation spatiale** : grille 2D régulière de **width** colonnes par **height** lignes de pixels.
- **Quantification de l'intensité** : chaque pixel prend une valeur entière dans $[0; 255]$ ($2^8 = 256$ nuances). 0 = noir, 255 = blanc.

Le POURQUOI : Choix des types de base (<stdint>)

- **Pourquoi uint8_t pour les pixels ?** Un int occupe 4 octets. Pour une image 1920×1080, utiliser int gaspillerait 6 Mo de mémoire vive ! Un uint8_t stocke exactement 1 octet dans $[0; 255]$.
- **Pourquoi uint16_t pour les dimensions ?** Sur 16 bits non signés, l'intervalle est $[0; 65\,535]$. Idéal pour toutes les images courantes, sans risque de valeurs négatives.

Structure en mémoire d'un pixel

1 octet (8 bits) = uint8_t

0 (Noir pur)

128 (Gris moyen)

255 (Blanc pur)

Pas de couleur (RGB) ni de canal alpha (transparence) dans ce TP1 : une seule valeur scalaire d'intensité par point.

Implantation « à plat » d'un tableau 2D

Principe et Schéma du cours

CM Diapo 25 / 87

		0	1	2	3	4	x
0		0	0	10	0	0	
y 1		10	10	15	10	10	
2		0	0	10	0	0	

(a)

(b)

- **Principe** : Mettre toutes les lignes bout à bout pour fabriquer un tableau mono-dimensionnel continu.
- La taille totale allouée est exactement de $\text{width} * \text{height}$ octets.
- L'accès au pixel (x, y) se fait à l'indice $t = y \times \text{width} + x$.

Le POURQUOI : Localité de Cache et Rapidité I/O

Si on utilisait `uint8_t**`, chaque ligne serait dispersée en mémoire. En 1D continu, les pixels sont consécutifs : le processeur précharge la ligne en cache L1/L2, et l'écriture disque se fait en **un seul appel système**.

Classe GrayImage (1/3) : Déclaration & Interdictions

Classe GrayImage : données membres, constructeurs et destructeur

CM Diapo 26 / 87

```
class GrayImage { // N'oubliez pas <cstdint> !
    const uint16_t width, height; // Largeur et hauteur de l'image dans [0; 65535].
    uint8_t * array;               // Pointeur vers le tableau alloué dynamiquement des pixels.
                                   // Chaque pixel peut varier dans [0; 255].

    GrayImage() = delete;          // Construction sans paramètre interdite ! (C++11)
public:
    GrayImage& operator=(const GrayImage& b) = delete; // Affectation interdite !
    GrayImage(uint16_t w, uint16_t h); // Construction à la dimension w * h
    GrayImage(const GrayImage& orig);   // Construction par copie
    ~GrayImage();                      // Destructeur
    ...
};
```

Le POURQUOI : Justification des contraintes C++11 (Slide 26)

- **Les dimensions resteront constantes pendant toute la vie de l'instance** ⇒ **pas de ré-allocation du tableau de pixels**. Cela garantit l'intégrité de la mémoire.
- **Pourquoi interdire la construction sans paramètre (= delete) ?** Une image sans taille n'a aucune existence concrète.
- **Pourquoi interdire l'opérateur d'affectation (= delete) ?** Une affectation `a = b` nécessiterait de changer la taille de `a`, ce qui est impossible car `width` et `height` sont `const` !

Constructeurs et Destructeur

CM Diapo 27 / 87

```
GrayImage::GrayImage(uint16_t w, uint16_t h) // Construction à partir des dimensions
    : width(w), height(h), array(nullptr) // Liste d'initialisation des données membres
{
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions strictement positives");
    array = new uint8_t[width * height];
}
GrayImage::GrayImage(const GrayImage& o) // Construction de copie
    : width(o.width), height(o.height), array(nullptr)
{
    array = new uint8_t[o.width * o.height]; // Allocation du nouveau tableau
    for (size_t t = 0; t < size_t(width * height); t++)
        array[t] = o.array[t]; // Recopie des valeurs des pixels
}
GrayImage::~GrayImage() { delete[] array; } // Destruction du tableau de pixels
```

Le POURQUOI : Tas (Heap) vs Pile (Stack) et Non-initialisation

- **Allocation dynamique sur le tas** : sa grande taille ferait exploser la **pile (stack)** !
- **Pourquoi ne pas initialiser les pixels dans le constructeur ?** M. Rémy le précise textuellement : « On choisit de ne pas initialiser les pixels... car on le fera avec d'autres fonctions ultérieurement » (gain de temps CPU).

Accesseurs de consultation et de modification

CM Diapo 28 / 87

```
inline const uint16_t& getWidth() const { return width; }
inline const uint16_t& getHeight() const { return height; }
// Accesseur de modification : retourne une référence modifiable (Left-Value)
inline uint8_t& pixel(uint16_t x, uint16_t y) {
    if (x >= width || y >= height) throw std::out_of_range("Hors limites");
    return array[y * width + x];
}
// Accesseur de consultation : retourne une référence constante
inline const uint8_t& pixel(uint16_t x, uint16_t y) const {
    if (x >= width || y >= height) throw std::out_of_range("Hors limites");
    return array[y * width + x];
}
```

Le POURQUOI : Inlining & Appel en Left-Value (C++ pur)

- **Déclaration « inline »** : évite d'empiler des contextes d'appel à chaque pixel dans une boucle de millions d'itérations. Le calcul d'indice est inséré directement.
- **Appel en Left-Value (spécifique au C++)** : Grâce au renvoi par référence `uint8_t&`, la méthode peut être placée à gauche d'une affectation : `im.pixel(10, 20) = 255;` modifie directement la case mémoire sans avoir besoin de `setPixel`.

Méthodes de Dessin : clear, rectangle & fillRectangle

clear(uint8_t gray = 0)

TP1 Question 12

```
void GrayImage::clear(uint8_t gray) {  
    for (size_t t = 0; t < size_t(width * height); t++)  
        array[t] = gray;  
}
```

Parcourt séquentiellement tout le tableau 1D. La valeur par défaut est 0 (noir absolu).

fillRectangle(x, y, w, h, gray)

TP1 Question 14

```
for (uint16_t j = 0; j < h; ++j)  
    for (uint16_t i = 0; i < w; ++i)  
        pixel(x + i, y + j) = gray;
```

Remplit toute la surface ligne par ligne (double boucle).

rectangle(x, y, w, h, gray) : Contour 1 pixel

TP1 Question 13

```
if (x + w > width || y + h > height)  
    throw std::out_of_range("Rectangle hors limites");  
// Lignes horizontales haut et bas  
for (uint16_t i = 0; i < w; ++i) {  
    pixel(x + i, y) = gray;  
    pixel(x + i, y + h - 1) = gray;  
}  
// Lignes verticales gauche et droite  
for (uint16_t j = 0; j < h; ++j) {  
    pixel(x, y + j) = gray;  
    pixel(x + w - 1, y + j) = gray;  
}
```

Le POURQUOI : Épaisseur & Bornes

Pour un cadre d'1 pixel d'épaisseur partant de (x, y), la dernière colonne est à x + w - 1 et la dernière ligne à y + h - 1. Tracer les 4 côtés directement est beaucoup plus rapide que de scanner toute la surface !

Structure du Format PGM (Portable Gray Map)

Spécification du Format P5 (Binaire)

TD1 Page 1/2

- 1. **Magic Number** : Chaîne "P5" + saut de ligne (LF / \n).
- 2. **Commentaires** : Lignes débutant par '#'.
Image sauvegardee... \n
- 3. **Dimensions** : Largeur puis hauteur en base 10 (ex: 320 240) + \n.
- 4. **Commentaires intercalaires** : Possible après les dimensions !
- 5. **Valeur maximale (dynamique)** : "255" en base 10 + 1 séparateur.
- 6. **Données pixels** : Exactement $w * h$ octets binaires consécutifs.

Schéma du flux de fichier PGM

```
P5\n# Image sauvegardee... \n320 240\n255\n
```

En-tête ASCII

Données Binaires (Pixels)

$320 \times 240 = 76\,800$ octets bruts

Hybridation texte/binaire (Slide 58) : L'en-tête est lisible en clair, tandis que le corps binaire est compact et directement injectable en mémoire.

Écriture PGM : void GrayImage::writePGM(...) const

Méthode d'instance writePGM

TD1 Question 2 & corrigé_TD.txt

```
void GrayImage::writePGM(std::ostream& os) const {
    os << "P5\n";
    os << "# Image sauvegardee par un etudiant pour les TP de RepCod.\n";
    os << width << " " << height << "\n";
    os << 255 << "\n";
    os.write((const char*)array, width * height); // Cast direct (const char*) vu en cours
}
```

Le POURQUOI : std::ostream& et os.write(...) avec cast (const char*)

- **Pourquoi std::ostream& et non std::ofstream& ?** C'est le principe de polymorphisme des flux : cette fonction peut écrire indifféremment dans un fichier disque (ofstream), sur la sortie standard (std::cout), ou dans un flux en mémoire (std::ostringstream) !
- **Pourquoi os.write(...) et le cast (const char*) ?** L'opérateur << sert au formatage ASCII (il traduirait les octets en nombres textuels). La méthode os.write envoie directement les **octets binaires bruts** du tableau sans aucune conversion. Comme os.write attend un const char*, M. Rémy utilise le cast direct (const char*)array vu au tableau.

Saut des Commentaires & Le Piège de chat.pgm

skip_line & skip_comments

TD1 Questions 3 & 4

```
void GrayImage::skip_line(std::istream& is) {
    char c;
    while (is.get(c) && c != '\n') {}
}

void GrayImage::skip_comments(std::istream& is) {
    char c;
    while (is.get(c)) {
        // Saut élémentaire des espaces et retours chariot
        if (c == ' ' || c == '\t' || c == '\n' || c == '\r')
            continue;
        if (c == '#')
            skip_line(is);
        else {
            is.putback(c); // Caractère utile retrouvé
            break;
        }
    }
}
```

Le Piège Mortel de chat.pgm Démasqué

Dans chat.pgm, des commentaires sont insérés **après** les dimensions :

```
320 240\n
# Encore du bordel\n
255\n
```

Quand on exécute `is >> w >> h`, le curseur s'arrête juste après le 240, c'est-à-dire **sur le \n** !

Si `skip_comments` ne saute pas les caractères blancs (`\n`), elle lit `\n`, voit que ce n'est pas `'#'`, et s'arrête. Ensuite, `is >> dyn` bute sur le `'#'` et **échoue** !

Lecture PGM : static GrayImage* readPGM(...)

Fonction de classe (static) readPGM

TD1 Question 5 & corrigé_TD.txt

```
GrayImage* GrayImage::readPGM(std::istream& is) {
    std::string magic;
    if (!(is >> magic) || (magic != "P5" && magic != "P2"))
        throw std::runtime_error("Magic number invalide (doit etre P5 ou P2)");
    skip_comments(is);
    uint16_t w, h; if (!(is >> w >> h)) throw std::runtime_error("Erreur dimensions");
    skip_comments(is);
    int dyn; if (!(is >> dyn) || dyn != 255) throw std::runtime_error("Dynamique invalide");
    GrayImage* res = new GrayImage(w, h); // Instanciation dynamique à la taille lue !
    if (magic == "P5") {
        char sep; is.get(sep); // Consomme l'unique caractère séparateur (\n) après 255
        is.read((char*)res->array, w * h); // Cast direct (char*) exact vu au TD
    } else { /* Boucle de lecture ASCII pour P2 */ }
    return res;
}
```

Le POURQUOI : Pourquoi une fonction STATIC ? (Le Serpent qui se mord la queue)

Comme le souligne M. Rémy : « Si nous plaçons readPGM comme membre d'instance, nous allons nous retrouver dans la situation paradoxale consistant à avoir besoin de disposer déjà d'une image pour pouvoir charger un fichier et créer une image : c'est le serpent qui se mord la queue ! »

Une méthode static s'appelle sur la classe sans instance préalable : `GrayImage* img = GrayImage::readPGM(f)`; tout en ayant accès aux membres privés (array).

Protocole de Test & Validation Visuelle (main.cpp)

Les 3 Épreuves de Contrôle du Sujet

TP1 Sujet

- **Épreuve 1 (Écriture & Géométrie)** : Créer une image **non-carrée** (ex: 400×250), effacer avec `clear(180)`, tracer des rectangles avec `rectangle()` et `fillRectangle()`, écrire avec `writePGM()`.
- **Épreuve 2 (Lecture & Modification)** : Charger `chat.pgm` avec `readPGM()`, dessiner dessus, puis sauvegarder sous un autre nom (`chat_modifie.pgm`).
- **Épreuve 3 (Exceptions)** : Vérifier qu'un appel `pixel(999, 999)` lève bien `std::out_of_range`.

Contrôle visuel sous Linux

Le professeur recommande d'utiliser **eog** ou **qiv** (Quick Image Viewer) :

```
$ g++ -std=c++11 -Wall -Wextra GrayImage.cpp main.cpp -o tp1
$ ./tp1
$ eog chat.pgm chat_modifie.pgm &
```

- En ouvrant les deux images dans la visionneuse, basculer avec les **flèches gauche/droite** ou la **barre d'espace** : la moindre anomalie saute aux yeux !
- « Évitez de lancer Gimp ou Photoshop : c'est super lourd et sans intérêt ici ! » (M. Rémy).

Les 5 Commandements du Code Rémy

- **Nommage exact** : Respecter à la lettre les prototypes du sujet (noté par scripts de test automatiques).
- **Indexation 1D** : Toujours maîtriser la formule $y * \text{width} + x$.
- **RAII & Mémoire** : Tout `new[]` a son `delete[]` dans le destructeur. Tout objet dynamique retourné par `readPGM` est libéré par l'appelant.
- **Exceptions C++** : Filtrer rigoureusement coordonnées et flux corrompus.

Rappels Stratégiques pour les Contrôles

- **Contrôle Théorique TD (14/10/2026)** : Maîtriser le découpage binaire, la table ASCII, et l'explication du rôle de `putback` et des méthodes `static`.
- **Contrôle Pratique TP (06/11/2026)** : 50% du travail fait chez soi ! Venir avec une classe `GrayImage` rodée et fonctionnelle pour répondre directement aux questions du contrôle sans perdre de temps.

Annexe Code (1/4) : GrayImage.hpp (Intégral)

Fichier d'en-tête GrayImage.hpp complet (syntaxe 100% cours)

Header C++

```
#ifndef GRAYIMAGE_HPP
#define GRAYIMAGE_HPP
#include <stdint.h>
#include <iostream>
#include <stdexcept>
#include <string>

class GrayImage {
private:
    const uint16_t width, height;
    uint8_t * array;
    GrayImage() = delete; // Construction sans paramètre interdite ! (C++11)

public:
    GrayImage& operator=(const GrayImage& b) = delete; // Affectation interdite !
    GrayImage(uint16_t w, uint16_t h);
    GrayImage(const GrayImage& orig);
    ~GrayImage();

    inline const uint16_t& getWidth() const { return width; }
    inline const uint16_t& getHeight() const { return height; }

    inline uint8_t& pixel(uint16_t x, uint16_t y) {
        if (x >= width || y >= height) throw std::out_of_range("Pixel hors limites");
        return array[y * width + x];
    }
    inline const uint8_t& pixel(uint16_t x, uint16_t y) const {
        if (x >= width || y >= height) throw std::out_of_range("Pixel hors limites");
        return array[y * width + x];
    }
}

void clear(uint8_t gray = 0);
void rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint8_t gray);
void fillRectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint8_t gray);
void writePGM(std::ostream& os) const;

static void skip_line(std::istream& is);
static void skip_comments(std::istream& is);
static GrayImage* readPGM(std::istream& is);
};
#endif // GRAYIMAGE_HPP
```

Annexe Code (2/4) : GrayImage.cpp (Mémoire & Dessin)

GrayImage.cpp : Constructeurs, Destructeur & Dessin

Partie 1/2

```
#include "GrayImage.hpp"

GrayImage::GrayImage(uint16_t w, uint16_t h) : width(w), height(h), array(nullptr) {
    if (w == 0 || h == 0) throw std::invalid_argument("Dimensions strictement positives");
    array = new uint8_t[width * height];
}

GrayImage::GrayImage(const GrayImage& o) : width(o.width), height(o.height), array(nullptr) {
    array = new uint8_t[o.width * o.height];
    for (size_t t = 0; t < size_t(width * height); t++) array[t] = o.array[t];
}

GrayImage::~GrayImage() { delete[] array; }

void GrayImage::clear(uint8_t gray) {
    for (size_t t = 0; t < size_t(width * height); t++) array[t] = gray;
}

void GrayImage::rectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint8_t gray) {
    if (w == 0 || h == 0) return;
    if (x + w > width || y + h > height) throw std::out_of_range("Rectangle hors limites");
    for (uint16_t i = 0; i < w; ++i) { pixel(x + i, y) = gray; pixel(x + i, y + h - 1) = gray; }
    for (uint16_t j = 0; j < h; ++j) { pixel(x, y + j) = gray; pixel(x + w - 1, y + j) = gray; }
}

void GrayImage::fillRectangle(uint16_t x, uint16_t y, uint16_t w, uint16_t h, uint8_t gray) {
    if (w == 0 || h == 0) return;
    if (x + w > width || y + h > height) throw std::out_of_range("Rectangle plein hors limites");
    for (uint16_t j = 0; j < h; ++j)
        for (uint16_t i = 0; i < w; ++i) pixel(x + i, y + j) = gray;
}
```

Annexe Code (3/4) : GrayImage.cpp (Entrées/Sorties PGM)

GrayImage.cpp : Écriture et Lecture PGM (syntaxe TD1 exacte)

Partie 2/2

```
void GrayImage::writePGM(std::ostream& os) const {
    os << "P5\n# Image sauvegardee pour les TP de RepCod.\n" << width << " " << height << "\n255\n";
    os.write((const char*)array, width * height); // Cast direct (const char*) exact du prof
}

void GrayImage::skip_line(std::istream& is) {
    char c; while (is.get(c) && c != '\n') {}
}

void GrayImage::skip_comments(std::istream& is) {
    char c;
    while (is.get(c)) {
        if (c == ' ' || c == '\t' || c == '\n' || c == '\r') continue; // Saut des blancs
        if (c == '#') skip_line(is);
        else { is.putback(c); break; }
    }
}

GrayImage* GrayImage::readPGM(std::istream& is) {
    std::string magic = ""; if (!(is >> magic)) throw std::runtime_error("Magic number absent");
    if (magic != "P5" && magic != "P2") throw std::runtime_error("Format non PGM");
    skip_comments(is);
    uint16_t w = 0, h = 0; if (!(is >> w >> h)) throw std::runtime_error("Erreur dimensions");
    skip_comments(is);
    int dyn = 0; if (!(is >> dyn) || dyn != 255) throw std::runtime_error("Dynamique != 255");
    GrayImage* res = new GrayImage(w, h);
    if (magic == "P5") {
        char sep; is.get(sep); // Consomme le séparateur (
    ) après 255
        is.read((char*)res->array, w * h); // Cast direct (char*) exact vu au TD
        if (!is) { delete res; throw std::runtime_error("Erreur lecture pixels"); }
    } else {
        for (size_t t = 0; t < size_t(w * h); ++t) {
            int val; if (!(is >> val)) { delete res; throw std::runtime_error("Erreur ASCII P2"); }
            res->array[t] = (uint8_t)val;
        }
    }
    return res;
}
```

Annexe Code (4/4) : main.cpp (Tests & Validation)

main.cpp : Scénario complet des 3 épreuves de test

Point d'entrée de test

```
#include "GrayImage.hpp"
#include <iostream>
#include <fstream>

int main() {
    try {
        // Test 1 : Image non carrée, clear(180), rectangles et sauvegarde writePGM
        GrayImage testImg(400, 250);
        testImg.clear(180);
        testImg.rectangle(20, 20, 100, 60, 0); // Cadre noir 1px
        testImg.fillRectangle(150, 40, 80, 80, 50); // Gris foncé plein
        testImg.fillRectangle(270, 50, 60, 120, 255); // Blanc plein
        std::ofstream fTest("test_rectangles.pgm", std::ios::binary);
        testImg.writePGM(fTest);

        // Test 2 : Lecture de chat.pgm, modifications géométriques et sauvegarde
        std::ifstream fChat("chat.pgm", std::ios::binary);
        if (!fChat) throw std::runtime_error("Impossible d'ouvrir chat.pgm");
        GrayImage* chat = GrayImage::readPGM(fChat);
        chat->rectangle(10, 10, 60, 40, 255);
        chat->fillRectangle(chat->getWidth() - 70, 10, 50, 50, 0);
        std::ofstream fChatOut("chat_modifie.pgm", std::ios::binary);
        chat->writePGM(fChatOut);
        delete chat; // Libération mémoire indispensable !

        // Test 3 : Gestion d'erreur out_of_range
        try { testImg.pixel(999, 999); }
        catch (const std::out_of_range& e) { std::cout << "Exception OK: " << e.what() << std::endl; }
        std::cout << "TOUS LES TESTS SONT VALIDES !" << std::endl;
    } catch (const std::exception& e) { std::cerr << "ERREUR: " << e.what() << std::endl; return 1; }
    return 0;
}
```